UNIVERSITÀ DEGLI STUDI DI BRESCIA

DIPARTIMENTO DI INGEGNERIA DELL'INFORMAZIONE

Corso di Laurea Magistrale in Ingegneria Informatica

Relazione Conclusiva

Elaborato del corso di Algoritmi e Strutture Dati

Studenti:
Bagnalasta Federico Matricola n. 735695
Kovacic Matteo Matricola n. 736588
Zambelli Samuele Matricola n. 736800

Anno Accademico 2024/2025

Indice

1 Introduzione **2**
1.1 Costrutti fondamentali 4

2 Compito 1 : Creazione della Griglia **8**

3 Compito 2: Calcolo di Contesto, Complemento e Frontiera **11**

4 Compito 3: Determinazione del Cammino Minimo Base **16**
4.1 Stop anticipato e Gestione dei Risultati Sub-Ottimi 19

5 Presentazione del Risultato **20**
5.1 Ottimizzazioni dell'Algoritmo Cammino Minimo 21

6 Compito 4: Analisi Prestazioni **24**
6.1 Analisi Spaziale 24
6.2 Analisi Temporale 25
6.3 Benchmark e Campione di Test 26
6.4 Analisi prestazioni 26
6.4.1 Presentazione delle scelte tecniche 26
6.4.2 Analisi tempi di esecuzione 27
6.4.3 Analisi utilizzo memoria 30
6.4.4 Celle di frontiera visitate 33
6.4.5 Analisi limiti della versione ottimizzata 37

7 Conclusioni **39**

8 Manuale d'installazione e utilizzo **42**
8.1 Requisiti di Sistema 42
8.2 Avvio del Programma 42
8.3 Sintassi generale 42
8.4 Modalità Batch (CLI) 43
8.5 Output della Modalità Batch 44
8.6 Modalità Grafica (GUI) 47
8.7 Salvataggio e Caricamento 49
8.8 Ottimizzazioni dell'Algoritmo 49

1 Introduzione

La presente relazione descrive lo sviluppo di un'applicazione dedicata alla risoluzione dei requisiti presentati dall'elaborato del corso di Algoritmi e Strutture Dati a.a. 2024-2025. Dopo un'approfondita analisi del problema e la definizione dei requisiti funzionali e non funzionali — già trattate nella documentazione fornita durante il corso — questa relazione intende fornire una sintesi delle scelte progettuali e implementative adottate, con particolare attenzione agli aspetti algoritmici, strutturali e prestazionali del sistema.

In primo luogo vengono illustrati i costrutti utilizzati per rappresentare la griglia, i nodi e le relazioni tra celle, motivando tali scelte alla luce delle esigenze di efficienza e manutenibilità. Successivamente, si discutono le principali strategie implementative adottate, evidenziando le soluzioni progettuali impiegate per ottimizzare l'esecuzione degli algoritmi di ricerca e di gestione dei percorsi.

La relazione prosegue con un'analisi delle ottimizzazioni introdotte sia a livello algoritmico sia a livello architetturale, seguita da una valutazione delle prestazioni ottenute mediante misurazioni sperimentali e confronto tra differenti configurazioni.
Infine, viene presentato il manuale d'uso dell'applicazione, volto a chiarire le modalità operative, l'interfaccia fornita e le principali funzionalità disponibili.

L'obiettivo complessivo è fornire una visione chiara, strutturata e motivata dell'intero processo di progettazione e realizzazione del sistema, consentendo al lettore di comprendere sia le scelte tecniche effettuate sia i risultati conseguiti.

Sintesi dei requisiti del progetto

Il sistema oggetto della presente applicazione opera su uno spazio bidimensionale discretizzato in celle, all'interno del quale è possibile definire percorsi tra celle attraversabili. Le celle sono considerate adiacenti secondo il modello delle otto direzioni (cardinali e diagonali), con costi unitari associati alle mosse cardinali e costi pari a $\sqrt{2}$ per quelle diagonali. Le celle non attraversabili possono comporre diverse tipologie d'ostacoli, come descritto dalla consegna.

Il progetto richiede innanzitutto la realizzazione di un generatore di griglie finite, configurabile dall'utente, in grado di produrre distribuzioni casuali di celle non attraversabili e diverse tipologie strutturate di ostacoli (semplici, aggregati, diagonali, delimitatori di aree chiuse, a barre e ulteriori conformazioni opportune). Tali griglie costituiscono la base dati per la successiva fase algoritmica.

Una seconda parte del lavoro riguarda lo sviluppo degli algoritmi necessari a determinare, data una cella origine, il relativo *contesto* e *complemento*, corrispondenti alle celle raggiungibili tramite cammini liberi di tipo 1 o di tipo 2. È inoltre richiesto il calcolo della distanza libera tra coppie di celle e l'individuazione di cammini minimi secondo

la definizione fornita, che include la possibilità di una costruzione ricorsiva mediante attraversamento delle celle di frontiera.

La terza componente del progetto consiste nell'estensione dell'applicazione affinché sia in grado di calcolare, se esiste, la sequenza di *landmark* di un cammino minimo da una cella origine a una cella destinazione, nonché il cammino completo corrispondente. L'elaborazione deve tenere conto della possibilità che la cella destinazione non appartenga alla chiusura dell'origine, rendendo necessaria la composizione di più cammini minimi intermedi. La consegna contempla inoltre la possibilità di introdurre criteri euristici di ordinamento delle celle di frontiera, finalizzati al miglioramento dell'efficienza computazionale.

Infine, il progetto prevede una fase di sperimentazione sistematica, basata sulle griglie generate al primo punto, finalizzata alla valutazione delle prestazioni dell'algoritmo `camminoMin()` e, ove pertinente, al confronto tra differenti strategie implementative ed euristiche. La sperimentazione deve essere condotta su tutte le tipologie di griglie prodotte dal generatore, così da garantire una valutazione completa e comparativa del comportamento dell'algoritmo al variare della struttura dello spazio di ricerca.

1.1 Costrutti fondamentali

I **costrutti principali** del progetto sono due: la **Cella** e la **Griglia**.
La **Cella** rappresenta il costrutto fondamentale del sistema ed è caratterizzata dai seguenti attributi:

- **riga** e **colonna**, che ne identificano la posizione all'interno della griglia;
- **tipo di cella**, che può assumere i valori di *ostacolo*, *complemento*, *contesto* oppure *attraversabile*;
- un **valore booleano** che indica l'appartenenza o meno della cella alla *frontiera*.

La **Griglia** è invece costituita da un **array bidimensionale di celle** e include ulteriori informazioni strutturali, quali la cella di **Origine**, quella di **Destinazione** e la **tipologia della griglia**.

Tipologie di Griglia

La tipologia della griglia è rappresentata dall'enumerazione `TipoPattern`, che consente di generare automaticamente diversi pattern di distribuzione degli ostacoli. Le tipologie supportate sono le seguenti:

- **DENSITÀ_BASSA**, **DENSITÀ_MEDIA**, **DENSITÀ_ALTA**: griglie generate automaticamente con una densità di ostacoli rispettivamente pari al **10%**, **25%** e **45%** del numero totale delle celle della griglia. Gli ostacoli vengono distribuiti in maniera casuale, secondo quanto descritto nel Capitolo 2.
- **PERSONALIZZATA**: griglia disegnata manualmente dall'utente tramite interfaccia grafica, che consente la massima libertà nella definizione della configurazione spaziale.
- **SPIRALE**, **SCACCHI**, **RIGHE**, **UNICO OSTACOLO**: pattern strutturati comunemente utilizzati, generati automaticamente dal sistema. Queste tipologie si sono rivelate particolarmente utili durante le fasi di individuazione e correzione degli errori nonché durante l'attività di validazione del software; per tale motivo, è stata scelta la loro inclusione tra le opzioni disponibili per l'utente.
- **VUOTA**: griglia priva di ostacoli, utile come caso limite per la verifica del corretto funzionamento degli algoritmi di ricerca del cammino.

Esempi di Pattern

Di seguito sono riportate alcune immagini illustrative delle diverse tipologie di griglia supportate dal sistema.

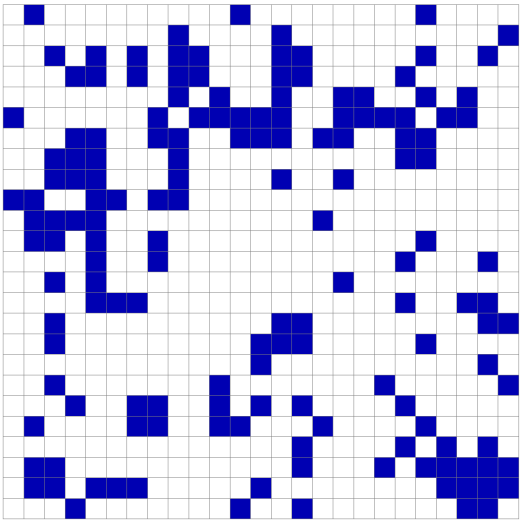


Figura 1: Esempio di griglia con pattern casuale a densità bassa.

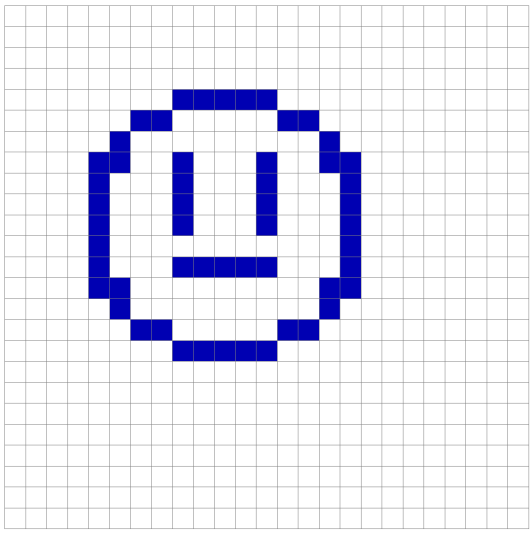
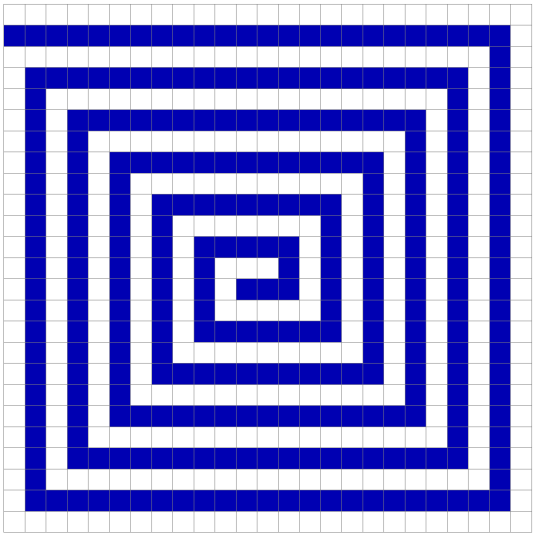
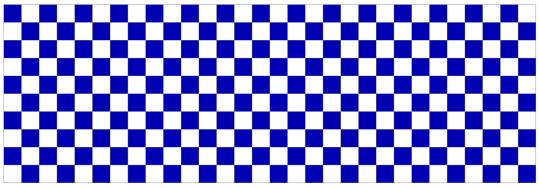


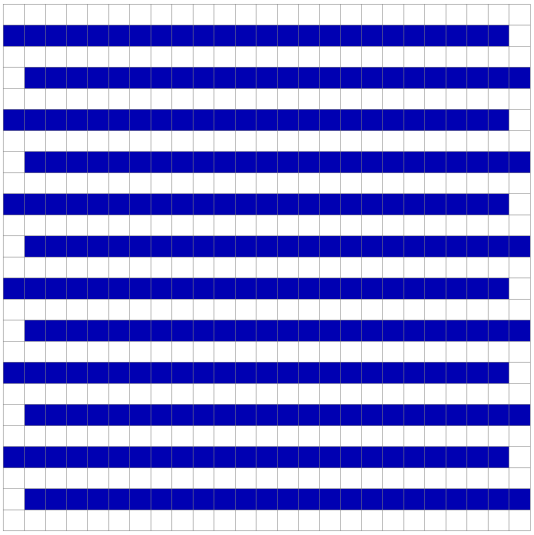
Figura 2: Esempio di griglia personalizzata definita dall'utente.



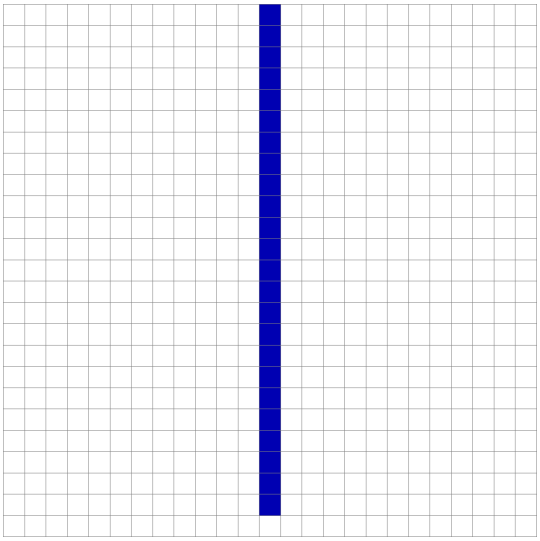
(a) Spirale



(b) Scacchi



(c) Righe



(d) Unico ostacolo

Figura 3: Esempi di pattern automatici.

All'interno del programma (eseguibile) è stata inoltre introdotta la possibilità di **serializzare l'oggetto Griglia in formato JSON**. Tale funzionalità consente di salvare e riutilizzare configurazioni precedenti, includendo implicitamente anche gli oggetti di tipo *Cella*. Questa scelta progettuale risulta particolarmente utile in fase di **testing**, poiché permette di memorizzare configurazioni specifiche di origine e destinazione, snellendo il processo di verifica. Inoltre, il riutilizzo delle stesse configurazioni favorisce la riproducibilità degli esperimenti e la coerenza dei risultati ottenuti.

Nota sulle dimensioni della griglia

L'oggetto **Griglia** può avere un numero arbitrario di righe e di colonne, non necessariamente uguale. Di conseguenza, la griglia non è vincolata a essere quadrata, ma può assumere forma rettangolare.Tuttavia, nella versione grafica, per motivi legati alle risorse di calcolo e alle prestazioni della macchina, è possibile creare una griglia con al massimo 10000 celle (100 righe $\times$ 100 colonne).

2 Compito 1 : Creazione della Griglia

La creazione della **Griglia** è gestita tramite un'**interfaccia grafica** che permette all'utente di:

- definire le **dimensioni della griglia**, che rappresentano i parametri in ingresso al costruttore della classe Griglia;
- scegliere un **pattern predefinito** tra quelli descritti nel Capitolo 1;
- inserire un **seme** per inizializzare il generatore casuale di griglie.

Al termine della fase di creazione, ciascuna delle celle che compongono la griglia può figurare come **ostacolo** o **attraversabile**.

Creazione dei Pattern di Ostacoli

La generazione dei pattern di ostacoli dipende dal tipo di pattern scelto:

- **Pattern deterministici:** SPIRALE, RIGHE, UNICO_OSTACOLO e SCACCHI. Per questi pattern, gli ostacoli vengono posizionati seguendo algoritmi deterministici che stabiliscono la posizione di ciascuna cella ostacolo.
- **Pattern casuali:** DENSITÀ_BASSA, DENSITÀ_MEDIA, DENSITÀ_ALTA. L'utente può selezionare la **densità di ostacoli** della griglia tra alcuni valori predefiniti (bassa densità, media densità o alta densità). Inoltre è possibile specificare un **seme** (numero intero) che determina la configurazione della griglia: se il seme è pari a 0, la disposizione delle celle varia a ogni esecuzione pur mantenendo invariata la densità; se il seme è diverso da 0, la configurazione generata rimane sempre la stessa. In particolare ciò che viene modificato è il **rapporto tra le varie tipologie di ostacoli base (Semplici, Agglomerati, Diagonali, Barre e Anelli)**. Oltre a questo il seme è anche usato per selezionare la **cella di partenza da cui generare ciascun ostacolo**.

Inoltre, il sistema permette all'utente di salvare in un **file .json** delle Griglie complete di ostacoli, Origine e Destinazione, che è possibile caricare in un secondo momento piuttosto di creare una griglia nuova.

Algoritmo di generazione degli ostacoli

Algorithm 1: generaOstacoli(Griglia, p, s)

Input: Griglia di dimensione $m \times n$, tipo di pattern p, (opzionale) seme s
Output: Griglia con celle ostacolo e attraversabili

```
if p in {SPIRALE, RIGHE, UNICO_OSTACOLO, SCACCHI} then
    // Algoritmo deterministico per il pattern scelto;
    PosizionaOstacoliDeterministicamente(Griglia, p);
else if p in {DENSITÀ_BASSA, DENSITÀ_MEDIA, DENSITÀ_ALTA} then
    InizializzaCasuale(s);
    rapporti=calcolaRapportiOstacoliBaseCasuale();
    calcolaQuantitàObiettivo(densità obiettivo, rapporti);
    ScegliCellaCasuale();
    PosizionaOstacoli();
    return Griglia;
```

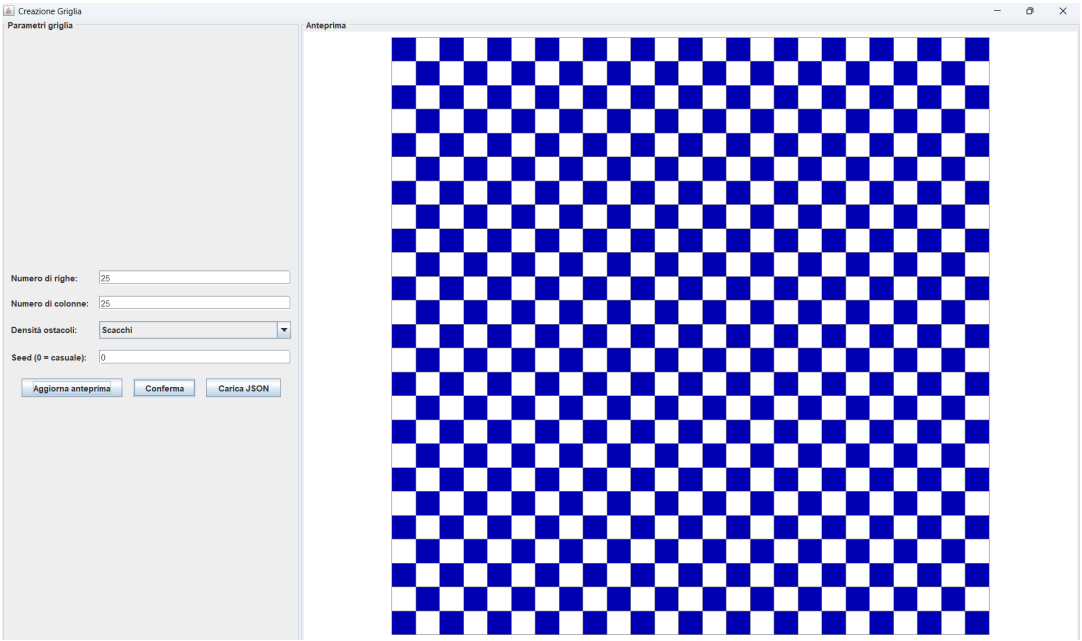


Figura 4: Esempi di generazione di griglia con pattern deterministico e casuale.

3 Compito 2: Calcolo di Contesto, Complemento e Frontiera

Una volta creata la griglia, l'utente accede a una schermata in cui è possibile selezionare una qualsiasi **cella attraversabile** per ottenere il **contesto** e il **complemento** di quella cella, evidenziati tramite due colori distinti.

Calcolo del Contesto

Fissata la **cella d'origine**, si scansionano tutte le celle della griglia per determinare quali appartengano al suo contesto, escludendo naturalmente le celle ostacolo.
Per questa operazione ci si affida all'algoritmo `camminoLibero(griglia, origine, destinazione, tipo)`, che funziona secondo la seguente logica:

1. Gestisce i casi banali: se **origine** e **destinazione** coincidono, la funzione restituisce **Vero**.
2. Controlla che origine e destinazione siano all'interno della griglia e non siano ostacoli; in caso contrario restituisce **Falso**.
3. Divide la griglia in quattro quadranti per stabilire la direzione verso la destinazione.
4. Imposta prima il **movimento diagonale** e successivamente quello assiale (orizzontale o verticale), verificando con la funzione `percorri()` se il cammino raggiunge la destinazione senza incontrare ostacoli o uscire dai bordi della griglia.
5. Se il movimento diagonale ha esito positivo, viene calcolato il movimento assiale rimanente in base alla direzione precedentemente determinata. In caso di esito negativo, la destinazione non appartiene al contesto.

Calcolo del Complemento

Il **complemento** viene calcolato solo per le celle che:

- sono **attraversabili**;
- non fanno già parte del contesto.

L'algoritmo utilizzato è analogo a quello del contesto, ma in questo caso il **movimento assiale** viene effettuato prima e quello **diagonale** dopo.

Pseudocodice degli algoritmi

Di seguito viene presentato lo pseudocodice dei due metodi principali:

Algorithm 2: camminoLibero(griglia, origine, destinazione, tipo)

```
Input: griglia, origine, destinazione, tipo (1=diagonale prima, 2=assiale prima)
Output: True se il cammino è libero, False altrimenti
dx = destinazione.colonna - origine.colonna;
dy = destinazione.riga - origine.riga;
if dx == 0 e dy == 0 then
    return True ;                                   // origine == destinazione
if origine o destinazione fuori griglia o ostacolo then
    return False
passoX = segno(dx);
passoY = segno(dy);
absDx = |dx|; absDy = |dy|;
passiDiagonali = min(absDx, absDy);
passiAssiali = |absDx − absDy|;
// movimento diagonale prima
if tipo == 1 then
    if percorri(griglia, origine, passoX, passoY, passiDiagonali) == False then
        return False;
    puntoCambio = percorriDiagonale(origine, passiDiagonali);     // cella da cui
     procedo in orizzontale
    if absDx >absDy then
        return percorri(griglia, puntoCambio, passoX, 0, passiAssiali);
    else
        return percorri(griglia, puntoCambio, 0, passoY, passiAssiali);
// movimento assiale prima
else if tipo == 2 then
    if absDx >absDy then
        if percorri(griglia, origine, passoX, 0, passiAssiali) == False then
            return False
        puntoCambio = percorriAssiale(origine, passiAssiali, orizzontale);
    else
        if percorri(griglia, origine, 0, passoY, passiAssiali) == False then
            return False
        puntoCambio = percorriAssiale(origine, passiAssiali, verticale);
    return percorri(griglia, puntoCambio, passoX, passoY, passiDiagonali)
```

Algorithm 3: percorri(griglia, origine, passoX, passoY, passi)

Input: griglia, origine (riga, colonna), passoX, passoY, passi
Output: True se il cammino è libero, False altrimenti

```
r, c = origine.riga, origine.colonna;
for i in 1..passi do
    r = r + passoX;
    c = c + passoY;
    if r o c fuori griglia then
        return False
    if griglia[r][c] è ostacolo then
        return False
return True
```

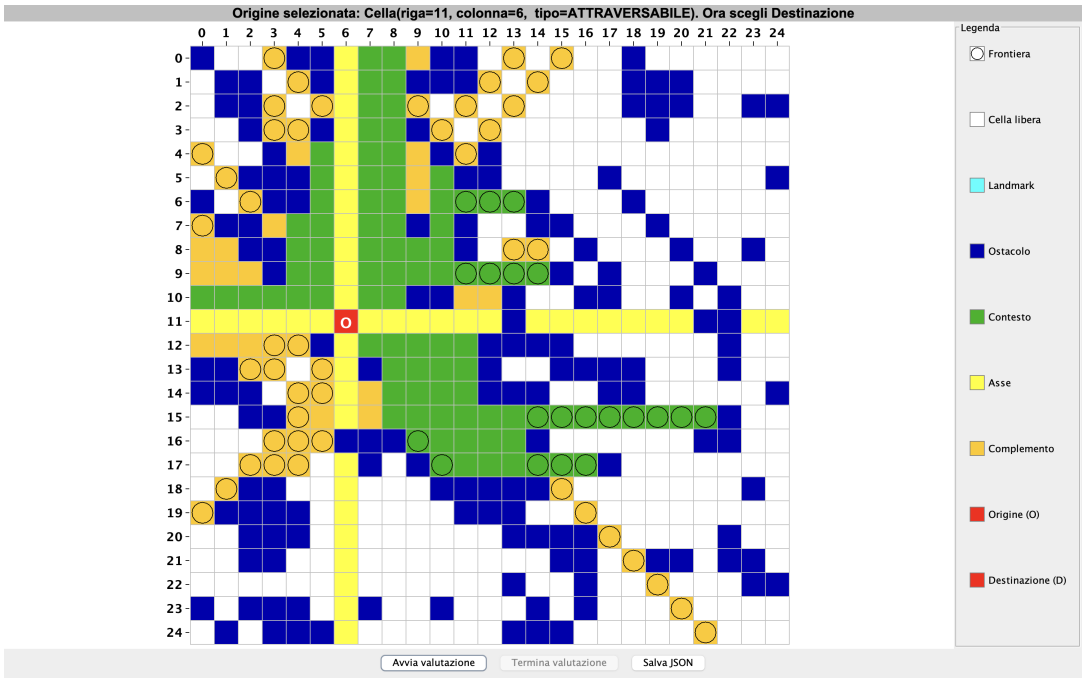


Figura 5: Esempio di evidenziazione del contesto (verde) e del complemento (arancione) di una cella O.

Riconoscimento delle Celle di Frontiera

Oltre al calcolo di contesto e complemento, è possibile individuare le **celle di frontiera** di una cella d'origine. Si definisce **frontiera** l'insieme delle celle appartenenti alla **chiusura**

dell'origine e adiacenti ad almeno una **cella attraversabile**.
Per riconoscerle, si procede nel seguente modo:

1. Si scansionano tutte le celle appartenenti alla chiusura dell'origine.
2. Per ciascuna cella c, si verifica se almeno una delle 8 celle adiacenti (verticali, orizzontali e diagonali) è di tipo *attraversabile*.
3. Se almeno una cella adiacente è attraversabile, la cella c viene contrassegnata come parte della **frontiera**.

Nota importante: il metodo presuppone che la cella c appartenga già alla chiusura (contesto o complemento) dell'origine.

Pseudocodice del metodo controllaVicini

Algorithm 4: controllaVicini(cella, griglia)

```
Input: cella (già appartenente alla chiusura dell'origine), griglia
Output: True se la cella è sulla frontiera, False altrimenti
// calcola le posizioni delle celle adiacenti come una combinazione di
   direzioni
for ogni direzione_x dx in {-1,0,1} do
    for ogni direzione_y dy in {-1,0,1} do
        if dx == 0 e dy == 0 then
            continua ;                                  // ignora la cella stessa
        nx = cella.colonna + dx;
        ny = cella.riga + dy;
        if nx o ny fuori griglia then
            continua
        if griglia[ny][nx] è attraversabile then
            return True
return False
```

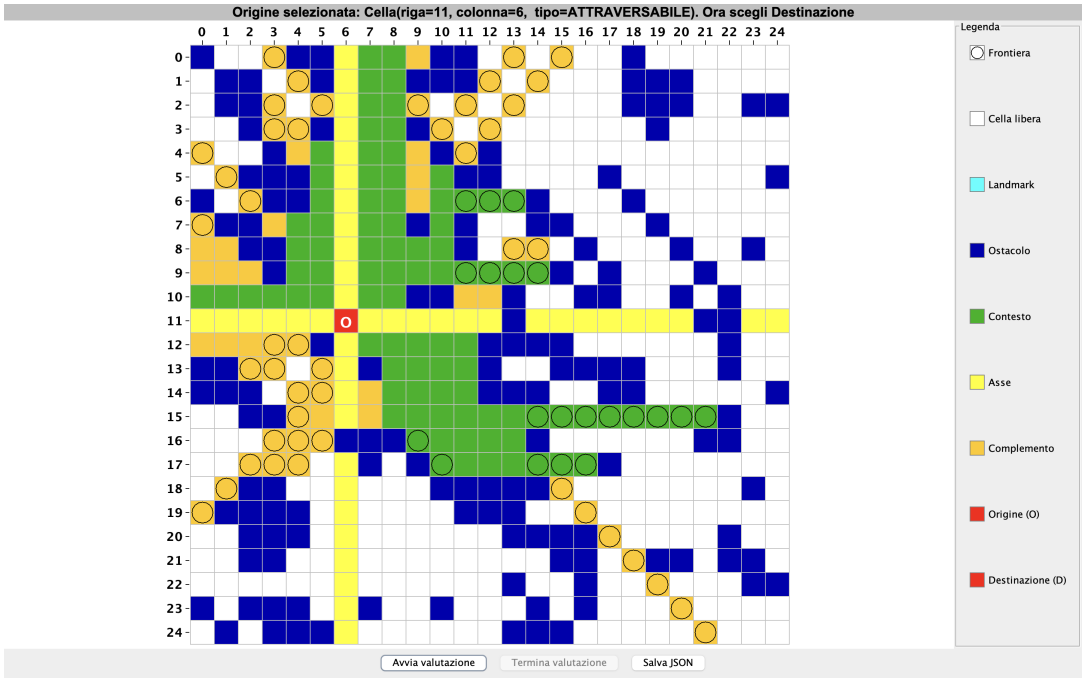


Figura 6: Esempio di evidenziazione delle celle di frontiera tramite un cerchio contenuto in alcune celle facenti parti del contesto o del complemento.

4 Compito 3: Determinazione del Cammino Minimo Base

Il **Compito Tre** si concentra sull'implementazione di un algoritmo per la determinazione del **Cammino Minimo** tra una cella **Origine** (O) e una cella **Destinazione** (D) all'interno di una griglia bidimensionale, che può contenere ostacoli. L'implementazione è gestita dalla classe `AnalizCam`, che espone il metodo principale `camminoMin`. Questo metodo media con l'utente e le altri classi attraverso una classe controller chiamata `AgenteCamminoMinimo`. CamminoMin gestisce la logica ricorsiva mentre l'agente si occupa dell'inizializzazione, delle statistiche, delle interruzioni e più in generale dell'adattamento delle classi esterne all'utilizzo del metodo `camminoMin`.

Pseudocodice dell'Algoritmo Cammino Minimo

Algorithm 5: camminoMin(griglia, O, D)

```
Input: griglia, origine O, destinazione D
Output: lunghezzaMinima, sequenza di Landmark
CONTROLLA_INTERRUZIONI();
CONTROLLA_UTILIZZO_MEMORIA();
camminoCorrente.aggiungi(O);
grigliaNuova ← CALCOLA_CC(griglia, O);
frontiera ← CALCOLA_FRONTIERA(grigliaNuova, O, D);
if D ∈ contesto[g] then
    aggiornaMigliorParziale();
    return DLIB(O, D), ⟨(O, 0), (D, 1)⟩;
if D ∈ complemento[g] then
    aggiornaMigliorParziale();
    return DLIB(O, D), ⟨(O, 0), (D, 2)⟩;
if frontiera = ∅ then
    return ∞, ⟨⟩;
lunghezzaMin ← ∞;
seqMin ← ⟨⟩;
RIORDINA_FRONTIERA();
foreach (F, t) ∈ frontiera do
    lF ← DLIB(O, F);
    if OTTIMIZZAZIONE_POSSIBILE() then
        continue;
    lFD, seqFD ← CAMMINO_MIN(grigliaNuova, F, D);
    lTot ← lF + lFD;
    if lTot < lunghezzaMin then
        lunghezzaMin ← lTot;
        seqMin ← COMPATTA(⟨(O, 0), (F, t)⟩, seqFD);
        camminoCorrente.rimuovi(ultimo);
return lunghezzaMin, seqMin;
```

Flusso di Esecuzione e Passi Fondamentali

La logica ricorsiva di `camminoMin` segue i seguenti passaggi principali:

1. **Preparazione della Ricorsione:** all'inizio di ogni chiamata ricorsiva, il livello di ricorsione viene incrementato e viene eseguito un controllo per eventuali interruzioni tramite `checkInterruzione`.

2. **Creazione della nuova configurazione della Griglia con Origine:** Questa griglia contiene la **chiusura** dell'origine (nuova), assegnando a ciascuna cella il valore di **Contesto** o **Complemento**.

3. **Caso Base – Destinazione Raggiunta:** l'algoritmo verifica se la cella destinazione appartiene al contesto o al complemento della cella origine. Se la condizione è soddisfatta, significa che la destinazione è direttamente raggiungibile dall'origine, e viene costruito e restituito un cammino dalla cella O alla cella D.

4. **Estrazione della Frontiera:** se il caso base non è soddisfatto, l'algoritmo individua la **frontiera** della griglia, ovvero l'insieme delle celle che rappresentano i possibili punti di espansione successivi.

5. **Caso Frontiera Vuota – Destinazione Irraggiungibile:** se la frontiera è vuota, significa che non esistono celle esplorabili dalla posizione corrente per raggiungere la destinazione. In questo scenario, il cammino è considerato irraggiungibile e viene restituita una lunghezza infinita con una sequenza di Landmark vuota.

6. **Esplorazione Ricorsiva della Frontiera:** per ogni cella F nella frontiera:

   - la griglia corrente viene copiata e modificata; la chiusura della cella corrente viene aggiunta come ostacolo per prevenire cicli. Questa griglia (grigliaNuova) viene utilizzata nei calcoli successivi;
   - viene calcolata nuovamente la frontiera;
   - viene applicata una **condizione di ottimizzazione** per decidere se procedere con la ricorsione su F, potando i rami di ricerca che non possono migliorare la lunghezza minima trovata fino a quel momento;
   - se la condizione è soddisfatta, viene effettuata la chiamata ricorsiva a `camminoMin(grigliaNuova, F, D)`, ottenendo il cammino minimo dalla cella F alla destinazione nella nuova griglia;
   - la lunghezza del cammino parziale lF viene sommata alla lunghezza ricorsiva lFD. Se la somma $lTot$ è minore della lunghezza minima corrente, sia `lunghezzaMin` sia `seqMin` vengono aggiornate. In caso di cammini con lunghezza identica, viene mantenuto quello scoperto per primo.

7. **Restituzione del Risultato:** al termine del ciclo su tutte le celle della frontiera, l'algoritmo restituisce il cammino minimo trovato e il livello di ricorsione viene decrementato.

8. **Fine Esecuzione:** il calcolo si conclude quando, al primo livello di ricorsione, tutte le celle di frontiera esplorabili sono state valutate. Il risultato finale rappresenta il **cammino minimo** tra Origine e Destinazione.

4.1 Stop anticipato e Gestione dei Risultati Sub-Ottimi

In alcuni scenari, l'algoritmo del Cammino Minimo potrebbe non completare la sua esecuzione entro il tempo disponibile o potrebbe essere interrotto dall'utente. Per gestire questi casi, è possibile implementare un meccanismo di **terminazione anticipata** che consente di visionare la migliore soluzione trovata fino a quel momento.

Concetto Chiave

A causa della natura **ricorsiva** dell'algoritmo, il cammino ottimo dall'origine alla destinazione viene costruito solo alla terminazione della procedura. Tuttavia, è possibile mantenere una copia dei **cammini sub-ottimi** ogni volta che l'algoritmo raggiunge la destinazione, confrontandoli con il cammino migliore precedentemente trovato. In questo modo, anche se l'algoritmo viene interrotto, si può restituire il cammino minimo *sub-ottimo* esplorato fino a quel momento.

Meccanismo di Salvataggio Intermedio

Il cammino corrente viene aggiornato dinamicamente durante l'esplorazione della griglia:

- ogni volta che si visita una nuova cella, questa viene aggiunta in coda: **cammino-Corrente.aggiungi(O)**;
- quando si risale l'albero dei percorsi (backtracking), l'ultima cella viene rimossa dalla coda: **camminoCorrente.rimuovi(ultimo)**;
- ogni volta che si raggiunge la cella destinazione, il cammino corrente viene confrontato con il **risultato intermedio** salvato: **aggiornaMigliorParziale()**;
- se la lunghezza del cammino corrente è minore di quella del risultato intermedio, quest'ultimo viene aggiornato.

Pseudocodice per la Gestione dei Risultati Sub-Ottimi

Algorithm 6: SalvataggioRisultatoIntermedio(camminoCorrente)

Input: camminoCorrente: lista di celle visitate, risultatoIntermedio: cammino migliore finora
Output: Aggiornamento di risultatoIntermedio se camminoCorrente è migliore

```
somma ← 0;
for i = 0 to camminoCorrente.size()-2 do
    somma ← somma + dlib(camminoCorrente[i], camminoCorrente[i+1]);
if somma < risultatoIntermedio.lunghezza then
    risultatoIntermedio.lunghezza ← somma;
    copiaLista(camminoCorrente, risultatoIntermedio.seq);
```

Integrazione nell'Algoritmo Base

Per gestire un'interruzione improvvisa, la classe che espone l'algoritmo principale (`camminoMin`) deve contenere la variabile globale descritta prima.
Vantaggi: questa strategia permette di ottenere un cammino sub-ottimo valido anche se l'algoritmo non ha completato tutte le esplorazioni, rendendo l'implementazione più robusta in scenari real-time o con vincoli di tempo.

5 Presentazione del Risultato

Il risultato prodotto dall'algoritmo del Cammino Minimo è rappresentato da un oggetto di tipo **RisultatoCammino**.
Per il corretto funzionamento dell'algoritmo, sono fondamentali soltanto due informazioni:

- la **lunghezza** totale del cammino minimo trovato;
- la **sequenza di celle** che costituisce il percorso dall'Origine alla Destinazione.

Tutti gli altri campi presenti nell'oggetto, come ad esempio il tempo di calcolo, la griglia di riferimento o informazioni sulla convergenza, rappresentano dati **aggiuntivi** utili per la visualizzazione, l'analisi e la reportistica. Queste informazioni vengono integrate nel risultato in una fase successiva, esterna all'algoritmo principale, e servono principalmente a fornire all'utente una visione più completa delle prestazioni e dello stato del calcolo.
Per una descrizione completa e dettagliata su come trattare, visualizzare e interagire con i risultati, si rimanda al **manuale d'uso del software**, che fornisce tutte le indicazioni operative e le modalità di interpretazione dei dati aggiuntivi.

5.1 Ottimizzazioni dell'Algoritmo Cammino Minimo

L'algoritmo del Cammino Minimo può essere significativamente migliorato introducendo alcune ottimizzazioni che ne riducono i tempi di esecuzione e aumentano l'efficienza complessiva. Le principali strategie implementate sono le seguenti.

Condizione più stringente per la potatura

La prima ottimizzazione consiste nel rafforzare la condizione alla riga 16 dello pseudocodice originale. Invece di utilizzare la condizione:

$$l_F > lunghezzaMin$$

si utilizza:

$$l_F + \mathrm{dLib}(F, dest) > lunghezzaMin$$

Questa modifica permette di potare preventivamente rami dell'albero dei percorsi che non possono produrre un cammino più breve, riducendo drasticamente lo spazio di ricerca.

2. Riordino della Frontiera

Un'altra ottimizzazione rilevante riguarda l'**ordinamento delle celle di frontiera** prima di iterare su di esse. Questa funzionalità è gestita dalla funzione `riordinaFrontiera`. Le celle vengono ordinate in base alla distanza libera (dlib), calcolata tra ciascuna di loro, e la destinazione del cammino.

riordinaFrontiera(List<Cella> frontiera, Cella destinazione)

Impatto: esplorare prima le celle più vicine alla destinazione aumenta le probabilità di trovare rapidamente un cammino minimo, riducendo le chiamate ricorsive non necessarie.

Algorithm 7: RiordinaFrontiera()

Input: frontiera: lista di celle, destinazione: cella D
Output: frontiera ordinata

```
frontiera.sort(Comparator.comparingDouble(l → dlib(l, destinazione)));
return frontiera
```

Nota: in uno sviluppo futuro si potrebbe utilizzare una struttura dati più efficiente (ad esempio una hash map con ID univoco per cella) per ridurre lo spazio di memoria riservato, mantenendo allo stesso tempo un costo temporale di accesso costante.

Rafforzamento della condizione globale

La terza ottimizzazione consiste nel rendere il confronto della lunghezza minima **globale**, anziché circostanziale al livello di ricorsione. In pratica, invece di confrontare solo la lunghezza parziale del cammino dal nodo corrente, si calcola una nuova lunghezza totale ogni volta che si raggiunge la destinazione. Questo approccio permette di potare in modo più aggressivo rami che non possono migliorare la lunghezza minima globale trovata fino a quel momento. È molto semplice da calcolare: basta includere nella chiamata dell'algoritmo la lunghezza migliore trovata fino a quel momento. Perciò, la prima invocazione diventa: `CamminoMin(griglia, origine, destinazione, 0)`, mentre quella ricorsiva diventa: CamminoMin(griglia, origine, destinazione, lunghezzaCorrente + dLib(origine, cellaFrontiera). Inoltre, è necessario adoperare una variabile (`lunghezzaMinGlobale`) che si trovi al di fuori della logica di ricorsione e che venga confrontata e, in caso aggiornata, ogni volta che si raggiunge la destinazione. Infine il confronto di riga 16 diventa:

$$lunghezzaCorrente + l_F + \text{dLib}(F, dest) < lunghezzaMinGlobale$$

Utilizzo della memoria dei nodi già visitati

Dato il comportamento ricorsivo dell'algoritmo per calcolare il Cammino Minimo, è possibile che lo stesso nodo venga visitato più volte durante una ricerca. Per evitare calcoli ridondanti, è stata implementata l'opzione per memorizzare il **miglior percorso trovato per ciascun nodo** in modo da potervi accedere in un secondo momento con un costo fisso. Per farlo si utilizza una struttura di supporto: un array bidimensionale `celleVisitateMatrice` che immagazzina il percorso da una cella qualsiasi a quella di destinazione salvando la sequenza indicizzata dalle coordinate della cella di origine della stessa.

Algorithm 8: Funzioni per la gestione della memorizzazione dei Cammini già Visitati

```
Funzione inizializzaMemoriaSequenze (nr, nc):
    celleVisitateMatrice ← nuovo array [nr][nc];
Funzione aggiungiAMemoria (cella, risultatoCammino):
    celleVisitateMatrice[cella.riga][cella.colonna] ← risultatoCammino;
Funzione checkMemoria (cella):
    return celleVisitateMatrice[cella.riga][cella.colonna];
```

Impatto: questa ottimizzazione riduce il numero di calcoli ricorsivi ridondanti, migliorando la velocità dell'algoritmo. Come `sviluppo futuro`, è possibile combinare le coordinate delle celle in un unico ID e utilizzare una **hash map** per ridurre lo spazio richiesto per la memorizzazione mantenendo un accesso diretto efficiente.

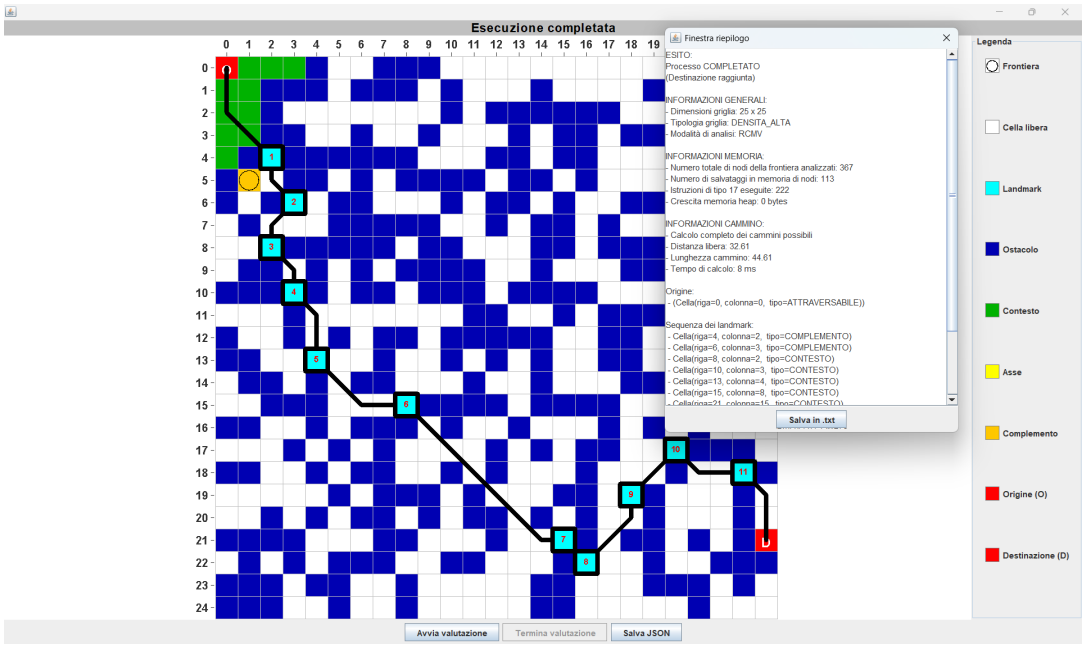


Figura 7: Esempio di visualizzazione grafica del percorso minimo generato da `CamminoMin` e a lato il riassunto con i dati relativi all'esecuzione.

6 Compito 4: Analisi Prestazioni

Si assuma che una Griglia abbia dimensione $m \times n$, dove m indica la larghezza e n l'altezza della Griglia. Le analisi che seguono sono state condotte considerando tutte le implementazioni del *Compito Tre*.

6.1 Analisi Spaziale

La firma del metodo principale accetta in ingresso una Griglia, una cella Origine e una cella Destinazione. La complessità spaziale iniziale risulta pertanto pari a:

$$O(m \cdot n) + O(1) + O(1) = O(m \cdot n)$$

Inoltre, l'algoritmo utilizza una variabile globale che opera come uno stack per mantenere il Cammino corrente sotto forma di sequenza di Landmark. Lo spazio massimo occupato in questo modo è proporzionale alla profondità massima l della ricorsione, contribuendo con un termine $O(l)$. E' inoltre utilizzata una struttura identica per salvare il miglior risultato della ricerca incompleta e quindi un'altra struttura di complessità $O(l)$.
All'inizio di ogni chiamata ricorsiva viene creata una nuova Griglia in cui è impostata la nuova Origine e la chiusura precedente viene trasformata in Ostacolo. Poiché le dimensioni della Griglia rimangono invariate, lo spazio totale occupato dalle Griglie generate durante le chiamate ricorsive è pari a $O(l) \cdot O(m \cdot n)$
Se l'algoritmo non termina immediatamente nel caso base, viene inoltre creata una struttura dati per contenere le Celle di Frontiera. Assumendo che, nel caso peggiore, tutte le Celle possano appartenere alla Frontiera, questa struttura contribuisce con $O(m \cdot n)$ che applicata a ogni nodo della ricorsione da una complessità di $O(l) \cdot O(m \cdot n)$.
Infine, bisogna considerare la complessità spaziale dovuta all'eventuale memorizzazione dei sottocammini parziali che permettono di calcolare nuovamente il percorso ricorsivamente per celle gia visitate. La sequenza è sicuramente contenuta nella griglia di dimensioni $m \cdot n$ e, nel caso peggiore, questa è riferita a un nodo trovato alla profondità l della ricorsione. Perciò, si tratta di un ulteriore $O(l \cdot m \cdot n)$.
Complessivamente, la complessità spaziale nel caso peggiore risulta:

$$2 \cdot O(l) + 3 \cdot O(l \cdot m \cdot n) = O(l \cdot m \cdot n)$$

Caso Migliore: Nel caso migliore, la Cella di Destinazione appartiene al Contesto o al Complemento dell'Origine. In questa situazione:

- lo stack contiene solo Origine e Destinazione;
- viene comunque creata una nuova Griglia di dimensione $O(m \cdot n)$;
- il Cammino risultante contiene soltanto due Landmark;

- la profondità massima della ricorsione è $l = 1$.

Pertanto, la complessità spaziale nel caso migliore è:

$$O(m \cdot n)$$

6.2 Analisi Temporale

La creazione di una nuova Griglia avviene tramite il costruttore della classe relativa, come già descritto nel *Compito Due*. Tale metodo utilizza un ciclo annidato sulla Griglia per creare un array bidimensionale di Celle. Questo richiede una complessità temporale pari a $O(m \cdot n)$.
Dopo aver creato la griglia con i relativi ostacoli, è possibile controllare se il cammino dall'Origine alla Destinazione è già stato memorizzato precedentemente. Questa verifica richiede di accedere a un array bidimensionale tramite le coordinate della Cella Origine con un tempo costante $O(1)$.
Dopo aver creato la griglia con i relativi ostacoli, è necessario considerare la complessità temporale richiesta per calcolare il Contesto, il Complemento e la Frontiera dell'Origine. Tutte e tre queste operazioni richiedono di effettuare un ciclo annidato sulla Griglia, ciascuno con un contributo pari a $O(m \cdot n)$ e quindi un totale di:

$$3 \cdot O(m \cdot n)$$

Ricorrere al riordinamento della frontiera come ottimizzazione richiede l'utilizzo della funzione **TimSort** che ha un costo temporale pari a $O(f \cdot log f)$, dove f è il numero di elementi presenti nella lista che contiene tutte le celle appartenenti alla frontiera. Nel nostro caso quindi il suo contributo è $O(m \cdot n \cdot log(m \cdot n))$.
Nel caso peggiore l'algoritmo per il Cammino Minimo viene applicato ricorsivamente l volte, una per ogni Cella di Frontiera su cui viene chiamato. Pertanto, la complessità spaziale nel caso peggiore è:

$$O(m \cdot n \cdot log(m \cdot n)) \cdot O(l)$$

Caso Migliore: Nel caso migliore, la Destinazione appartiene al Contesto o al Complemento dell'Origine. Di conseguenza, le operazioni necessarie sono la creazione della Griglia (con i relativi ostacoli) e il calcolo del Contesto e del Complemento dell'Origine. Perciò, la complessità temporale nel caso migliore (caso in cui la Destinazione faccia parte del Contesto dell'Origine) è:

$$2 \cdot O(m \cdot n) = O(m \cdot n)$$

6.3 Benchmark e Campione di Test

Per l'analisi delle prestazioni dell'algoritmo, è stato creato un insieme di griglie di test rappresentative dei diversi scenari possibili.
Le configurazioni principali includono:

- Griglie di diverse dimensioni (100×100, 25×25 e 10×10) e con densità di ostacoli variabile, per valutare l'impatto della scala e della complessità spaziale.
- Per ciascun pattern di Griglia predefinito è stata realizzata almeno una Griglia 25×25, consentendo di testare il comportamento dell'algoritmo nel caso in cui la disposizione degli ostacoli nella Griglia non è casuale.

In tutti i casi, i punti di **Origine** e **Destinazione** sono stati scelti in modo da massimizzare la difficoltà del percorso, garantendo test impegnativi per l'algoritmo. Ogni configurazione di griglia è stata inoltre duplicata invertendo Origine e Destinazione, per verificare la simmetria e la robustezza delle prestazioni rispetto alla direzione del cammino.

6.4 Analisi prestazioni

6.4.1 Presentazione delle scelte tecniche

Sebbene il campione di test includa griglie di dimensioni differenti, al fine di valutare l'impatto delle diverse modalità applicabili all'algoritmo `camminoMin()`, nelle analisi che seguono è stato scelto di considerare soltanto griglie di dimensione 25×25. Questa scelta consente di confrontare in modo diretto le diverse modalità dell'algoritmo su un insieme omogeneo di istanze, variando esclusivamente la disposizione e la densità degli ostacoli. In tal modo è possibile isolare l'effetto delle singole ottimizzazioni e delle loro combinazioni, evitando che le differenze osservate nei risultati siano influenzate prevalentemente dalla dimensione della griglia.

Le prestazioni dell'algoritmo sono state valutate considerando tre metriche principali: **tempo di esecuzione**, **utilizzo della memoria** e **numero di celle frontiera visitate**. Queste grandezze permettono di analizzare il comportamento dell'algoritmo da punti di vista complementari: il tempo di esecuzione misura l'efficienza computazionale complessiva, il numero di celle visitate fornisce una stima dell'ampiezza dello spazio di ricerca esplorato, mentre l'utilizzo della memoria evidenzia il costo delle strutture dati impiegate.
Per ciascuna configurazione di griglia, i test sono stati ripetuti abilitando selettivamente le diverse ottimizzazioni introdotte nell'algoritmo.

In particolare, i risultati riportati fanno riferimento alle seguenti modalità:

- **V: controllo della lunghezza globale del cammino**
- **M: memorizzazione delle sequenze già esplorate**
- **C: applicazione della condizione forte**
- **R: riordinamento della frontiera di esplorazione**

Oltre all'analisi delle singole ottimizzazioni, sono state considerate anche diverse combinazioni delle stesse, al fine di valutare eventuali effetti sinergici e identificare le configurazioni più efficaci. I risultati sono sempre confrontati con l'algoritmo base, privo di ottimizzazioni, utilizzato come riferimento.

6.4.2 Analisi tempi di esecuzione

In questa sezione sono riportati i tempi di esecuzione dell'algoritmo `camminoMin()` su tabelle 25×25, presentati in forma tabellare e grafica, e ne viene fornita una breve analisi.

tipologia_dimensione	—-	—V	–M-	-C–	R—
DENSITA ALTA 25x25	60000.375	9547.375	5.875	53189.125	60000.125
		(84.09%)	(99.99%)	(11.35%)	(0.0%)
DENSITA BASSA 25x25	45511.5	31.75	14.875	34477.375	45286.75
		(99.93%)	(99.97%)	(24.24%)	(0.49%)
DENSITA MEDIA 25x25	60000.3	6062.1	10.8	60000.0	60000.1
		(89.9%)	(99.98%)	(0.0%)	(0.0%)
RIGHE 25x25	95.0	65.5	0.5	85.0	98.5
		(31.05%)	(99.47%)	(10.53%)	(-3.68%)
SCACCHI 25x25	60000.0	16078.5	8.5	60000.0	60000.0
		(73.2%)	(99.99%)	(0.0%)	(0.0%)
SPIRALE 25x25	3.5	2.5	0.5	2.5	3.0
		(28.57%)	(85.71%)	(28.57%)	(14.29%)
UNICO OSTACOLO 25x25	0.0	0.0	0.0	0.0	0.5
		(nan%)	(nan%)	(nan%)	(-inf%)

tipologia_dimensione	R–V	R-MV	RC–	RCM-
DENSITA ALTA 25x25	129.125	2.625	45723.625	3.875
	(99.78%)	(100.0%)	(23.79%)	(99.99%)
DENSITA BASSA 25x25	15.5	2.25	15980.125	3.75
	(99.97%)	(100.0%)	(64.89%)	(99.99%)
DENSITA MEDIA 25x25	14.6	2.8	56459.3	6.6
	(99.98%)	(100.0%)	(5.9%)	(99.99%)
RIGHE 25x25	63.0	1.0	82.5	1.0
	(33.68%)	(98.95%)	(13.16%)	(98.95%)
SCACCHI 25x25	13753.0	2.5	60001.0	6.0
	(77.08%)	(100.0%)	(-0.0%)	(99.99%)
SPIRALE 25x25	4.0	1.0	2.0	1.0
	(-14.29%)	(71.43%)	(42.86%)	(71.43%)
UNICO OSTACOLO 25x25	0.0 (nan%)	0.0 (nan%)	0.0 (nan%)	0.0 (nan%)

Tabella 1: Tempi di esecuzioni medi misurati in ms per griglie 25×25 per diverse modalità di esecuzione di `CamminoMin()`

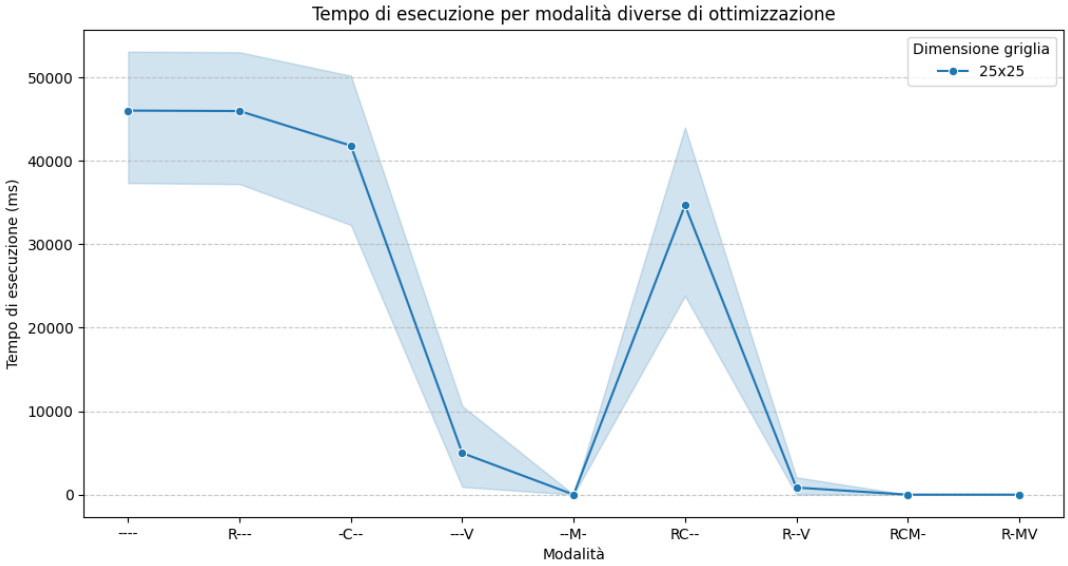


Figura 8: Tempi di esecuzione per griglie 25×25

Un primo risultato evidente riguarda la **memorizzazione delle sequenze** (M), che emerge come **l'ottimizzazione di gran lunga più efficace**. In tutte le configurazioni caratterizzate da una complessità non banale – in particolare nei casi di densità alta, media e bassa degli ostacoli, così come nel pattern a scacchiera – l'introduzione di M porta a una riduzione dei tempi di esecuzione prossima al 100%.
I tempi passano infatti da decine di secondi a pochi millisecondi, rendendo **l'impatto delle altre ottimizzazioni quasi trascurabile in confronto**. Questo comportamento conferma che la possibilità di evitare l'esplorazione ripetuta di sequenze già valutate riduce drasticamente lo spazio di ricerca effettivo, soprattutto in contesti in cui l'algoritmo tenderebbe naturalmente a ripercorrere cammini simili.

Il **controllo della lunghezza globale** (V) fornisce anch'esso un contributo significativo, sebbene meno marcato. Nei casi più complessi, come le griglie ad alta densità o con disposizione a scacchiera, V consente di ridurre i tempi di esecuzione in misura consistente, spesso superiore al 70–80%. Questo risultato indica che la potatura basata sulla lunghezza globale riesce a eliminare molte esplorazioni non promettenti quando **esiste un'elevata varietà di cammini alternativi**. Al contrario, in configurazioni più regolari o fortemente vincolate, come le griglie a righe o a spirale, il beneficio risulta più contenuto, poiché il numero di percorsi realmente alternativi è limitato fin dall'inizio.

La **condizione forte** (C) mostra invece un **impatto più discontinuo**. In alcune configurazioni, in particolare con densità alta o bassa, essa contribuisce a una riduzione

moderata dei tempi, mentre in altri casi – come la densità media o il pattern a scacchi – il suo effetto risulta praticamente nullo. Ciò suggerisce che questa ottimizzazione **entra in gioco solo in presenza di specifiche condizioni strutturali o se combinato ad altre tecniche quali il Riordino della frontiera** che consentono di interrompere anticipatamente l'esplorazione, mentre risulta poco incisiva in contesti molto simmetrici o fortemente vincolati.

Il **riordinamento della frontiera** (R), se considerato isolatamente, **non produce miglioramenti rilevanti** sui tempi di esecuzione. In molte configurazioni i tempi rimangono sostanzialmente invariati rispetto all'algoritmo base, mentre in alcuni casi si osservano variazioni minime o addirittura leggermente peggiorative.
Questo comportamento è riconducibile al **costo computazionale del riordinamento** stesso, che **non viene compensato** da una **riduzione significativa dello spazio di ricerca** quando manca un meccanismo più incisivo di potatura o memorizzazione.

L'analisi delle combinazioni di ottimizzazioni conferma ulteriormente questi risultati.
Tutte le configurazioni che includono la memorizzazione delle sequenze mostrano **tempi di esecuzione estremamente ridotti** e sostanzialmente **indipendenti dalla tipologia di griglia**. In particolare, combinazioni come R–MV o RCM- garantiscono riduzioni prossime al 100% in tutti gli scenari testati.
Al contrario, le combinazioni che non includono M **migliorano solo parzialmente le prestazioni** e rimangono **fortemente influenzate dalla disposizione degli ostacoli**.

Infine, il caso limite rappresentato dalla griglia con un unico ostacolo produce **tempi di esecuzione nulli** o **trascurabili** per tutte le configurazioni, rendendo le percentuali di miglioramento non significative. Questo scenario, essendo **computazionalmente banale**, **non fornisce indicazioni utili** sul contributo relativo delle diverse ottimizzazioni.

Nel complesso, l'analisi dei tempi di esecuzione evidenzia come la memorizzazione delle sequenze sia il **fattore determinante** per la scalabilità dell'algoritmo, mentre le altre ottimizzazioni svolgono un **ruolo di supporto**, efficace soprattutto se integrate in **strategie combinate**.

6.4.3 Analisi utilizzo memoria

In questa sezione è riportato l'utilizzo della memoria da parte dell'algoritmo `camminoMin()` su tabelle 25×25, presentati in forma tabellare e grafica, e ne viene fornita una breve analisi.

tipologia_dimensione	—-	—V	–M-	-C–	R—
DENSITA ALTA 25x25	166413.625	71541.75 (57.01%)	4096.0 (97.54%)	176719.875 (-6.19%)	182594.5 (-9.72%)
DENSITA BASSA 25x25	155137.875	9796.25 (93.69%)	5709.5 (96.32%)	124962.5 (19.45%)	120578.625 (22.28%)
DENSITA MEDIA 25x25	156214.4	42877.7 (72.55%)	2582.5 (98.35%)	184059.3 (-17.82%)	130258.8 (16.62%)
RIGHE 25x25	61909.0	16900.0 (72.7%)	0.0 (100.0%)	76279.0 (-23.21%)	74291.0 (-20.0%)
SCACCHI 25x25	160776.5	109041.0 (32.18%)	4096.0 (97.45%)	175018.0 (-8.86%)	127813.5 (20.5%)
SPIRALE 25x25	2048.0	2048.0 (0.0%)	0.0 (100.0%)	2048.0 (0.0%)	2048.0 (0.0%)
UNICO OSTACOLO 25x25	0.0	0.0 (nan%)	0.0 (nan%)	0.0 (nan%)	0.0 (nan%)

tipologia_dimensione	R–V	R-MV	RC–	RCM-
DENSITA ALTA 25x25	46199.125 (72.24%)	2048.0 (98.77%)	120459.875 (27.61%)	4096.0 (97.54%)
DENSITA BASSA 25x25	2056.125 (98.67%)	18.875 (99.99%)	52572.5 (66.11%)	1551.75 (99.0%)
DENSITA MEDIA 25x25	13949.9 (91.07%)	835.0 (99.47%)	119415.9 (23.56%)	3732.0 (97.61%)
RIGHE 25x25	33234.5 (46.32%)	0.0 (100.0%)	86473.0 (-39.68%)	0.0 (100.0%)
SCACCHI 25x25	102922.0 (35.98%)	0.0 (100.0%)	156859.5 (2.44%)	4096.0 (97.45%)
SPIRALE 25x25	2048.0 (0.0%)	0.0 (100.0%)	2048.0 (0.0%)	0.0 (100.0%)
UNICO OSTACOLO 25x25	0.0 (nan%)	0.0 (nan%)	0.0 (nan%)	0.0 (nan%)

Tabella 2: Utilizzo della memoria medio (Heap) misurato in KB per griglie 25X25 per diverse modalità di esecuzione di `CamminoMIn`

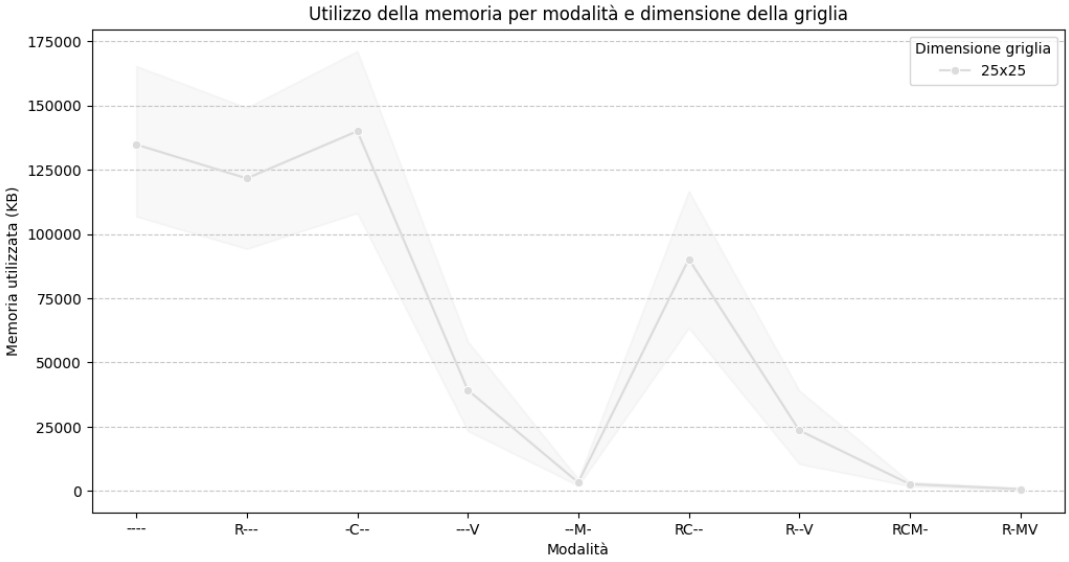


Figura 9: Memoria utilizzata per analizzare griglie 25x25

L'analisi dell'utilizzo della memoria mette in evidenza un comportamento in parte diverso rispetto a quanto osservato per i tempi di esecuzione, sottolineando come le ottimizzazioni introdotte influenzino non solo l'efficienza temporale, ma anche il costo in termini di risorse occupate.

Il risultato più significativo riguarda ancora una volta la **memorizzazione delle sequenze** (M), che si conferma l'**ottimizzazione più incisiva** anche dal punto di vista della memoria. In tutte le configurazioni non banali, l'introduzione di M comporta una **riduzione estremamente marcata** dell'utilizzo di memoria, spesso superiore al 97–99%. Questo risultato riveste particolare rilevanza poiché, nonostante l'impiego di una struttura aggiuntiva per la gestione della memoria, consente un'ottimizzazione dell'algoritmo che determina una riduzione della sua complessità, rendendolo complessivamente meno dispendioso in termini di risorse rispetto alle versioni prive di meccanismi di memoria.
In diversi casi, come nei pattern a righe, scacchi e spirale, il consumo di memoria si riduce addirittura a **valori nulli** o **prossimi allo zero**. Questo comportamento indica che la memorizzazione delle sequenze consente di eliminare gran parte delle strutture dati intermedie, limitando drasticamente il numero di stati che devono essere mantenuti in memoria.

Il **controllo della lunghezza globale** (V) produce anch'esso una **riduzione significativa dell'utilizzo di memoria**, sebbene meno uniforme rispetto a M. Nei casi di densità alta e media, V riduce il consumo di memoria di oltre il 70%, mentre in configurazioni

come la scacchiera l'effetto risulta più contenuto. Ciò suggerisce che la potatura basata sulla lunghezza globale contribuisce a limitare il numero di nodi attivi in memoria, ma la **sua efficacia dipende fortemente** dalla **struttura del problema**.

La **condizione forte** (C) mostra un impatto complessivamente **limitato** e, in alcuni casi, **addirittura negativo**.
In diverse configurazioni, l'attivazione di C comporta un **aumento dell'utilizzo di memoria** rispetto all'algoritmo base. Questo fenomeno può essere attribuito alla **necessità di mantenere informazioni aggiuntive** per verificare la condizione stessa, **senza** che ciò si traduca in una **riduzione proporzionale dello spazio di ricerca**.
Tali risultati evidenziano come un'**ottimizzazione efficace dal punto di vista logico non sia necessariamente vantaggiosa in termini di consumo di memoria**.

Il **riordinamento della frontiera** (R) presenta un comportamento analogo: se utilizzato isolatamente, **raramente porta a una riduzione significativa dell'utilizzo di memoria** e, in alcuni casi, **introduce un overhead** dovuto alle operazioni di gestione e riordinamento delle strutture dati.
Questo conferma che **il solo riordinamento non è sufficiente a contenere il consumo di memoria in assenza di meccanismi più incisivi di potatura**.

L'analisi delle combinazioni di ottimizzazioni rafforza ulteriormente queste osservazioni. Le configurazioni che includono la memorizzazione delle sequenze mostrano sistematicamente i **valori di utilizzo di memoria più bassi**, spesso prossimi allo zero, **indipendentemente dalla tipologia di griglia**.
In particolare, le combinazioni R–MV e RCM- risultano essere le più efficaci, garantendo una riduzione quasi totale dell'occupazione di memoria in tutti gli scenari analizzati.
Al contrario, le combinazioni prive di M presentano **riduzioni più modeste** e rimangono fortemente influenzate dalla disposizione degli ostacoli.
Infine, come già osservato per i tempi di esecuzione, il caso della griglia con un unico ostacolo rappresenta uno **scenario banale**, in cui l'utilizzo di memoria è nullo per tutte le configurazioni e **non fornisce indicazioni significative ai fini del confronto**.

6.4.4 Celle di frontiera visitate

In questa sezione è riportato il numero di celle di frontiera visitate dall'algoritmo `camminoMin()` su tabelle 25×25, presentati in forma tabellare e grafica, e ne viene fornita una breve analisi.

tipologia_dimensione	—-	—V	–M-	-C–	R—
DENSITA ALTA 25x25	4465894.5	802166.25 (82.04%)	863.0 (99.98%)	3820077.5 (14.46%)	4749064.0 (-6.34%)
DENSITA BASSA 25x25	3530459.0	19330.875 (99.45%)	4000.375 (99.89%)	2593580.5 (26.54%)	3732981.5 (-5.74%)
DENSITA MEDIA 25x25	4800263.2	609682.1 (87.3%)	2734.6 (99.94%)	5394353.0 (-12.38%)	4830160.9 (-0.62%)
RIGHE 25x25	8190.0	7222.0 (11.82%)	76.0 (99.07%)	7848.5 (4.17%)	8190.0 (0.0%)
SCACCHI 25x25	4681437.0	4406664.5 (5.87%)	1767.5 (99.96%)	4661318.5 (0.43%)	4093376.5 (12.56%)
SPIRALE 25x25	149.5	149.5 (0.0%)	54.5 (63.55%)	149.5 (0.0%)	149.5 (0.0%)
UNICO OSTACOLO 25x25	1.0	1.0 (0.0%)	1.0 (0.0%)	1.0 (0.0%)	1.0 (0.0%)

tipologia_dimensione	R–V	R-MV	RC–	RCM-
DENSITA ALTA 25x25	41104.75 (99.08%)	628.25 (99.99%)	4171490.625 (6.59%)	768.25 (99.98%)
DENSITA BASSA 25x25	13034.625 (99.63%)	1575.625 (99.96%)	1411685.0 (60.01%)	1909.375 (99.95%)
DENSITA MEDIA 25x25	8487.7 (99.82%)	1473.4 (99.97%)	5179700.6 (-7.9%)	2674.8 (99.94%)
RIGHE 25x25	7283.0 (11.07%)	75.0 (99.08%)	7507.0 (8.34%)	75.0 (99.08%)
SCACCHI 25x25	3673137.5 (21.54%)	1197.0 (99.97%)	4358836.5 (6.89%)	1557.0 (99.97%)
SPIRALE 25x25	149.5 (0.0%)	54.5 (63.55%)	149.5 (0.0%)	54.5 (63.55%)
UNICO OSTACOLO 25x25	1.0 (0.0%)	1.0 (0.0%)	1.0 (0.0%)	1.0 (0.0%)

Tabella 3: Celle di frontiera analizzate (media) per griglie 25X25 per diverse modalità di esecuzione di `CamminoMIn`

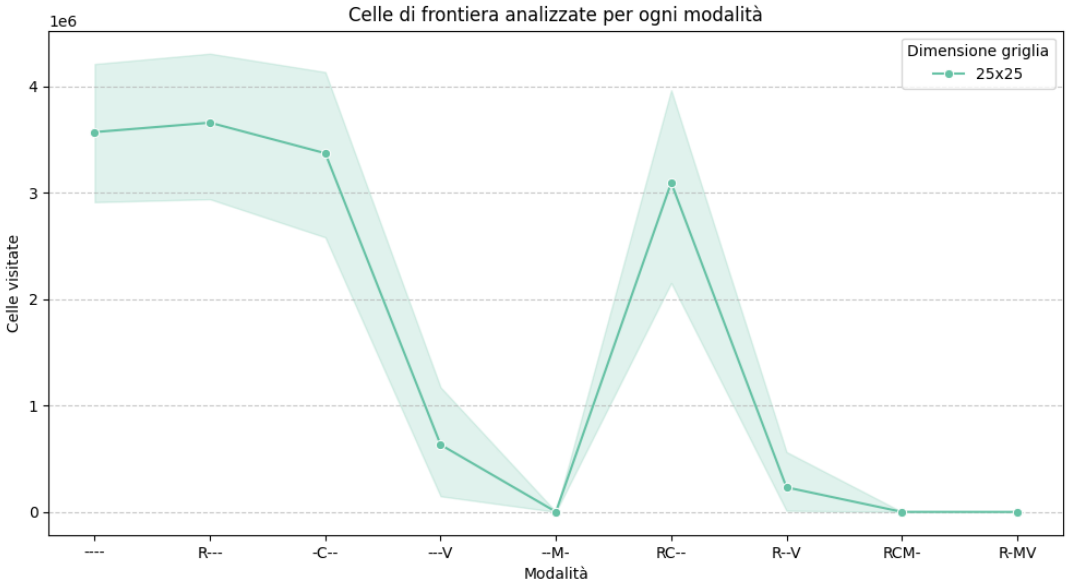


Figura 10: Celle di frontiera visitate per griglie 25x25

L'analisi del numero di celle di frontiera visitate evidenzia con particolare chiarezza l'effetto delle diverse ottimizzazioni sul comportamento esplorativo dell'algoritmo. A differenza del tempo di esecuzione, questa metrica permette di osservare direttamente quanto l'algoritmo riesca a contenere l'espansione dello spazio di ricerca.

Ancora una volta, la **memorizzazione delle sequenze** (M) emerge come l'**elemento più determinante**. In tutte le configurazioni caratterizzate da una complessità elevata, il numero di celle di frontiera visitate si riduce di oltre il 99%, passando da milioni di celle esplorate a poche migliaia, e in alcuni casi a poche centinaia. Questo risultato indica che la memorizzazione delle sequenze non solo riduce il tempo di esecuzione, ma **incide in modo diretto e sostanziale sulla strategia di esplorazione**, **evitando la riespansione di stati già analizzati** e **limitando drasticamente la crescita della frontiera**. I

Il **controllo della lunghezza globale** (V) mostra un comportamento **fortemente dipendente dalla struttura della griglia**. Nei casi di densità alta, media e bassa, V consente una riduzione significativa del numero di celle visitate, spesso superiore all'80% e in alcuni casi prossima al 99%.
Al contrario, in configurazioni più strutturate o simmetriche, come il pattern a scacchiera o quello a righe, l'effetto risulta molto più contenuto.

Ciò suggerisce che la potatura basata sulla lunghezza globale è **particolarmente efficace quando esiste un'ampia varietà di cammini alternativi**, mentre **perde incisività in scenari in cui i percorsi sono fortemente vincolati**.

La **condizione forte** (C) presenta un **impatto limitato** e **non sempre positivo**.
In diverse configurazioni, l'attivazione di C produce una **riduzione marginale del numero di celle frontiera visitate**, mentre in altri casi si osserva addirittura un **aumento rispetto all'algoritmo base**.
Questo comportamento indica che la condizione forte **non riesce sempre a ridurre lo spazio di ricerca** e, in alcune situazioni, **può introdurre vincoli che non si traducono in un'effettiva riduzione delle esplorazioni**.

Il **riordinamento della frontiera** (R), se utilizzato isolatamente, **non mostra benefici significativi** sul numero di celle visitate. In molti casi il **valore rimane sostanzialmente invariato** rispetto all'algoritmo base, mentre in alcune configurazioni si osservano **leggere variazioni**, **sia positive che negative**.
Questo risultato conferma che il riordinamento, da solo, **non è sufficiente a contenere l'espansione dello spazio di ricerca** senza un meccanismo di potatura più incisiva.

L'analisi delle combinazioni di ottimizzazioni rafforza ulteriormente queste osservazioni. Tutte le configurazioni che includono la memorizzazione delle sequenze mostrano una **riduzione estremamente marcata del numero di celle di frontiera visitate**, spesso superiore al 99%, **indipendentemente dalla tipologia di griglia**.
In particolare, le combinazioni R–MV e RCM- risultano tra le più efficaci, confermando che l'**effetto dominante è legato alla presenza di M**, mentre le altre ottimizzazioni svolgono un ruolo complementare.
Infine, i casi della spirale e della griglia con un unico ostacolo rappresentano scenari **fortemente vincolati** o **banali**, in cui il numero di celle visitate è già molto ridotto nell'algoritmo base. In questi casi, le ottimizzazioni producono **benefici limitati** o **nulli**, rendendo tali configurazioni **poco significative ai fini del confronto**.

6.4.5 Analisi limiti della versione ottimizzata

Dimensione	Numero celle (rc)	Tempo medio (ms)
10x10	100	0.17
25x25	625	2.31
100x100	10000	849.86
200x200	40000	6006.00
250x250	62500	17569.33
275x275	75625	29642.00
300x300	90000	44165.00
325x325	105625	46480.00
350x350	122500	60001.00

Tabella 4: Tempi di esecuizione in ms per la modalità di ottimizzazione migliore R-MV

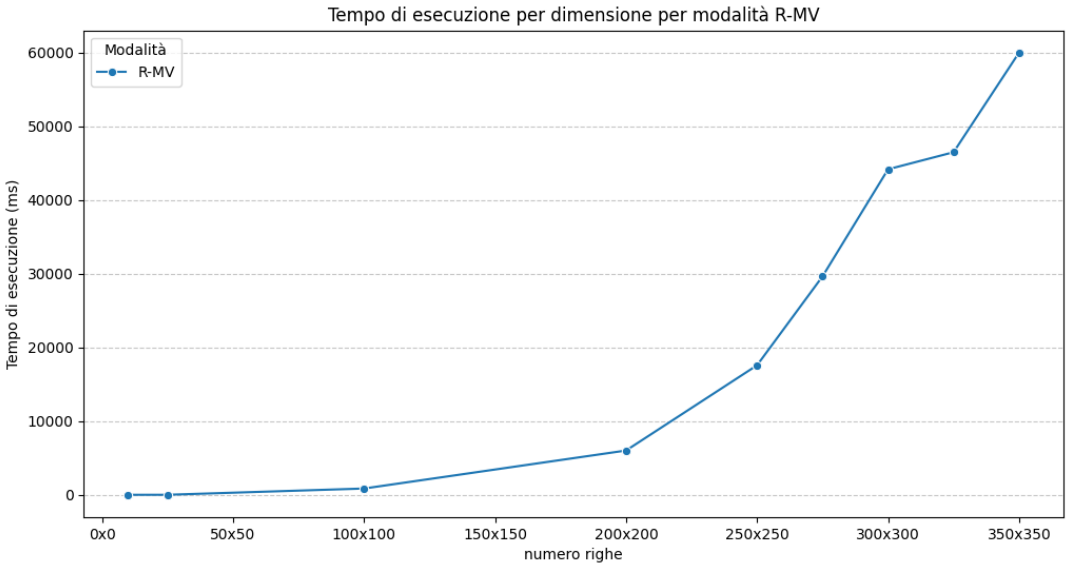


Figura 11: Rapppresentazione grafica della tabella 4

Come benchmark per l'ottenimento di questi risultati è stata utilizzata la Griglia con **pattern Scacchi** in modo da avere una complessità del percorso da trovare sfidante per l'algoritmo e allo stesso tempo essere in grado di determinare a priori se la Destinazione possa essere raggiunta.

La tabella mostra un **aumento non lineare** del tempo medio di esecuzione al crescere della dimensione della griglia.

Il tempo non cresce esattamente proporzionale al numero di celle, ma più rapidamente, soprattutto per griglie grandi.
Ad esempio:
Da 100×100 (10.000 celle) a 200×200 (40.000 celle), il numero di celle cresce di $4\times$, mentre il tempo passa da 849,86 ms a 6006 ms (circa $7\times$).
Da 250×250 a 300×300, le celle aumentano di circa $1,44\times$, da 62.500 a 90.000, ma il tempo quasi raddoppia, passando da 17.569 ms a 44.165 ms.
Questo suggerisce che l'algoritmo abbia complessità **super-lineare**, probabilmente tra

$$O(m \cdot n)^2 \text{ e } O(m \cdot n)^3$$

La densità alta di ostacoli del pattern a scacchiera, aumenta la complessità della ricerca dei percorsi.
Questo spiega l'aumento così rapido dei tempi per griglie grandi: più ostacoli ovvero più iterazioni necessarie per trovare il percorso minimo.
Per griglie fino a 100×100 celle, le prestazioni sono **accettabili** ($<$1s).
Per griglie maggiori di 200×200, invece, i tempi diventano **molto lunghi**.

Nel complesso, questa analisi mostra come le ottimizzazioni introdotte consentano di estendere in modo significativo il dominio di applicabilità dell'algoritmo rispetto alla versione base, rendendolo efficace su griglie di dimensioni medio-grandi. Tuttavia, l'andamento osservato conferma che, per griglie di dimensioni molto elevate, il **costo computazionale** associato alla ricerca del cammino minimo diventa **rapidamente proibitivo**, rendendo necessarie ulteriori strategie di ottimizzazione o un cambio di paradigma algoritmico per affrontare problemi su scala ancora maggiore.

7 Conclusioni

Il progetto presentato in questa relazione ha avuto come obiettivo lo studio e la valutazione sperimentale di un algoritmo per la ricerca di un percorso ottimo in una griglia con ostacoli, ponendo particolare attenzione all'impatto di diverse modalità operative sul comportamento complessivo dell'esplorazione. L'attenzione non è stata rivolta esclusivamente alla correttezza dell'algoritmo, ma soprattutto alla **comprensione delle conseguenze che specifiche scelte progettuali hanno sulle prestazioni**.

Attraverso un insieme di benchmark rappresentativi, costruiti variando densità della griglia e disposizione degli ostacoli, è stato possibile osservare come le diverse configurazioni influenzino in modo significativo il comportamento dell'algoritmo. In particolare, l'analisi condotta sulle griglie di dimensione 25×25 ha consentito di isolare l'effetto delle singole ottimizzazioni, permettendone un confronto chiaro e significativo.

I risultati sperimentali mostrano in modo consistente che la memorizzazione delle sequenze rappresenta l'ottimizzazione più incisiva. Essa risulta determinante non solo nella riduzione dei tempi di esecuzione, ma anche nel contenimento del numero di celle visitate e dell'utilizzo della memoria. Questo comportamento evidenzia come l'eliminazione delle riesplorazioni ridondanti incida direttamente sulla complessità pratica dell'algoritmo, rendendolo scalabile anche in scenari caratterizzati da un'elevata complessità combinatoria.

Il controllo della lunghezza globale si è dimostrato efficace soprattutto in configurazioni con un elevato numero di cammini alternativi, contribuendo a ridurre in modo significativo lo spazio di ricerca e, di conseguenza, il tempo di esecuzione. Tuttavia, la sua efficacia risulta fortemente dipendente dalla struttura della griglia, mostrando benefici più limitati in contesti fortemente vincolati o altamente simmetrici.

La condizione forte e il riordinamento della frontiera, se considerati isolatamente, presentano invece un impatto meno uniforme. In alcuni casi tali meccanismi introducono benefici marginali, mentre in altri possono addirittura peggiorare alcune metriche, come il numero di celle visitate o l'utilizzo della memoria. Questi risultati mettono in evidenza come una strategia di potatura più restrittiva non garantisca necessariamente una riduzione monotona dello spazio esplorato, ma possa influenzare l'ordine e la dinamica dell'esplorazione, con effetti non immediatamente intuitivi.

L'analisi congiunta delle tre metriche considerate ha inoltre evidenziato l'esistenza di trade-off tra tempo di esecuzione, spazio di ricerca e consumo di memoria. Ottimizzazioni particolarmente efficaci su una metrica non sempre producono miglioramenti equivalenti sulle altre, sottolineando l'importanza di una valutazione multidimensionale

delle prestazioni e la necessità di scegliere la configurazione dell'algoritmo in funzione del contesto applicativo.

Nel complesso, il lavoro svolto dimostra come l'integrazione di meccanismi di ottimizzazione adeguati possa **migliorare in modo sostanziale le prestazioni di un algoritmo di ricerca del cammino**, ma anche come tali meccanismi debbano essere analizzati con attenzione per comprenderne appieno gli effetti collaterali. I risultati ottenuti confermano la validità dell'approccio adottato e forniscono una base solida per eventuali sviluppi futuri.

Sviluppi futuri

Nel corso dello sviluppo del software, l'attenzione principale è stata rivolta alla riduzione del numero di nodi esplorati nell'albero di ricorsione, privilegiando strategie algoritmiche di potatura e ottimizzazione del processo di ricerca del cammino. Di conseguenza, sono stati volutamente tralasciati **approcci orientati allo sfruttamento diretto delle risorse hardware**, che potrebbero costituire una naturale estensione del lavoro svolto.

In particolare, non sono state investigate tecniche di ottimizzazione legate alle prestazioni della macchina, quali l'**uso esplicito della memoria cache**, la **parallelizzazione delle computazioni** o l'impiego di **strutture dati e algoritmi progettati per favorire la località dei dati**. L'introduzione di tali strategie potrebbe portare a un miglioramento significativo dei tempi di esecuzione, soprattutto nel caso di griglie di grandi dimensioni o di alberi di ricerca particolarmente profondi.

Analogamente, l'ottimizzazione dello spazio occupato non è stata oggetto di un'analisi approfondita. Attualmente, la memorizzazione dei nodi già visitati avviene tramite una matrice bidimensionale indicizzata direttamente dalle coordinate della Griglia. Sebbene questa soluzione risulti semplice ed efficiente in termini di accesso, essa è potenzialmente sovradimensionata dal punto di vista della memoria utilizzata. Un possibile sviluppo futuro consiste nella sostituzione di tale struttura con una ***hash map***, ottenuta combinando le coordinate della Cella in un identificatore univoco. Questo approccio consentirebbe di **ridurre sensibilmente l'occupazione di memoria**, mantenendo al contempo un accesso diretto ai nodi già esplorati.

Un'ulteriore direzione di miglioramento riguarda la rappresentazione stessa della Griglia. L'utilizzo di strutture compatte, come ad esempio ***bitmap*** o rappresentazioni binarie, in luogo di una matrice di oggetti Cella, permetterebbe di **ridurre in maniera significativa l'ingombro in memoria**. Tale scelta risulterebbe particolarmente vantaggiosa nel caso di Griglie di grandi dimensioni e di alberi di ricorsione fortemente stratificati, nei quali il numero di strutture allocate cresce rapidamente.

Nel complesso, l'integrazione di tali ottimizzazioni consentirebbe di **migliorare in modo sistematico l'efficienza dell'algoritmo**, riducendo i tempi di esecuzione nelle istanze più complesse e **contenendo l'occupazione di memoria** richiesta dalle strutture dati impiegate durante l'esplorazione

8 Manuale d'installazione e utilizzo

Il presente documento costituisce il **manuale d'uso ufficiale** del software per l'analisi e il calcolo del **cammino minimo su griglie bidimensionali con ostacoli**.
Il sistema è progettato per:

- studiare algoritmi di ricerca del cammino minimo;
- confrontare differenti configurazioni e ottimizzazioni;
- analizzare le prestazioni in termini di **tempo** e **memoria**;
- supportare sia un utilizzo **interattivo (GUI)** sia **automatizzato (CLI)**.

Il software mette a disposizione:

- una **modalità batch** (CLI) per test massivi su griglie memorizzate in cartelle;
- una **modalità grafica** per creazione, modifica e visualizzazione delle griglie;
- strumenti di **reportistica** (CSV, TXT, JSON);

8.1 Requisiti di Sistema

- Java JDK 17 o superiore
- Sistema operativo: Windows, Linux o macOS
- Almeno 2 GB di RAM consigliati

8.2 Avvio del Programma

Il programma si presenta sottoforma di **file .jar**. Per avviare l'eseguibile è necessario eseguire da terminale il comando `java -jar ASD_24_25.jar`.

Il programma viene avviato tramite la classe `Main` e supporta due modalità mutuamente esclusive:

- **Modalità batch (CLI)**
- **Modalità grafica (GUI)**

8.3 Sintassi generale

```
java Main <opzione> [argomenti]
```

Opzioni disponibili

Opzione	Descrizione
`-b <directory>`	Avvia il programma in modalità CLI, leggendo tutte le griglie JSON presenti nella cartella indicata.
`-g`	Avvia il programma in modalità grafica (GUI).

8.4 Modalità Batch (CLI)

La modalità batch è progettata per eseguire **valutazioni automatiche su molte griglie** senza intervento manuale.

Avvio

```
java Main -b ./griglie/
```

Il programma:

1. legge tutti i file `.json` presenti nella cartella;
2. supporta sia singole griglie che array di griglie;
3. carica tutte le griglie valide in memoria;
4. avvia il controller CLI.

Formato dei file JSON

Ogni file può contenere:

- una singola griglia;
- un array di griglie.

Le griglie includono:

- dimensioni;
- matrice degli ostacoli;
- origine e destinazione (opzionali).

Inserimento dei Parametri

Durante l'esecuzione CLI, l'utente viene guidato nell'inserimento dei parametri:

Limite di tempo

- espresso in millisecondi;
- limite massimo di esecuzione per ogni griglia;
- se superato, l'algoritmo viene interrotto.

Stringa di opzioni booleane

L'utente inserisce una stringa di 4 caratteri:

`RCMV`

Ogni posizione rappresenta un'opzione:

Posizione	Simbolo	Significato
1	R	Riordino della frontiera
2	C	Condizione forte
3	M	Memoria delle sequenze
4	V	Versione ottimizzata

Il simbolo `-` indica disattivazione.
Nota importante: Il funzionamento interno e teorico di ciascuna opzione è descritto in dettaglio nella **Sezione 5.1**.
Una volta terminata l'esecuzione si entra in un secondo sotto menu che chiederà all'utente di decidere se continuare con una nuova esecuzione, oppure uscire.

8.5 Output della Modalità Batch

Report CSV

Al termine dell'esecuzione viene generato un resoconto contenuto nel file `report.csv`.

Interpretazione del file CSV generato

Il file `report.csv` contiene l'aggregato di tutti i risultati ottenuti durante l'analisi delle griglie caricate. Il documento è strutturato secondo il formato CSV (*Comma-Separated Values*) con intestazione e valori separati da virgola. Ogni riga del file rappresenta i risultati ottenuti per una singola griglia testata.
Il contenuto del CSV viene generato dal metodo `generaResoconto` della classe `ValutatoreGriglie`. L'intestazione del file contiene i seguenti campi:

1. **modalità**: indica la modalita di esecuzione dell'algoritmo, quali sono le ottimizzazioni attive. Si presenta nello stesso formato della richiesta dei parametri. Alcuni esempi: "—-" individua il caso base (senza ottimizzazioni), "R—" è il caso in cui è attiva soltanto l'ottimizzazione di riordino frontiera, "R-M-" indica che riordino frontiera e memorizzazione dei percorsi sono attivi, etc.

2. **righe**: indica il numero di righe della griglia testata. Questo valore è fondamentale per comprendere la dimensione verticale della griglia e, indirettamente, la complessità computazionale del calcolo del cammino minimo, poiché griglie più grandi richiedono un numero maggiore di iterazioni dell'algoritmo di ricerca.

3. **colonne**: rappresenta il numero di colonne della griglia. Analogamente al campo `righe`, fornisce informazioni sulla dimensione orizzontale della griglia. L'accoppiamento di righe e colonne consente di calcolare l'area complessiva della griglia, utile per confronti tra test su griglie di dimensioni diverse.

4. **tipologia**: specifica il tipo di configurazione della griglia, tra "DENSITA_BASSA", "DENSITA_MEDIA", "DENSITA_ALTA", "SPIRALE", "SCACCHI", "RIGHE", "UNICO_OSTACOLO", e "PERSONALIZZATA". Questo campo permette di distinguere tra griglie generate automaticamente secondo pattern predefiniti e griglie create manualmente dall'utente, e consente di correlare le prestazioni dell'algoritmo con la complessità della disposizione degli ostacoli.

5. **d_lib**: rappresenta la distanza tra Origine e Destinazione in una Griglia senza ostacoli, perciò è di fatto il limite inferiore per i valori che `Cammino Minimo` può assumere. Maggiore è la differenza tra queste due misure, maggiore è il numero di ostacoli presenti tra Origine e Destinazione.

6. **dest_ragg**: indica se la destinazione è raggiungibile dalla posizione di partenza nella griglia. Questo campo assume valori binari:

   - 1: la destinazione è raggiungibile.
   - 0: la destinazione non è raggiungibile a causa della disposizione degli ostacoli.

Questa informazione è essenziale per interpretare i risultati di convergenza e lunghezza del cammino: se la destinazione non è raggiungibile, il resto dei valori potrebbe risultare incompleto o non significativo.

7. **convergenza**: indica se l'algoritmo ha completato la ricerca del cammino minimo entro i vincoli temporali impostati. Il campo assume i valori:

   - 1: il calcolo ha raggiunto la convergenza completa.
   - 0: il calcolo è stato interrotto anticipatamente (per limite di tempo o per intervento dell'utente).

   Per maggiori dettagli sulle condizioni di interruzione e sulle ottimizzazioni utilizzate, si rimanda alla sezione 5.1.

8. **lunghezza**: rappresenta la lunghezza del cammino minimo trovato dall'agente. Questo valore numerico indica il numero di passi necessari per raggiungere la destinazione dalla posizione di partenza, considerando il percorso calcolato dall'algoritmo. Il campo è espresso in numeri interi o decimali, a seconda del tipo di calcolo implementato.

9. **tempo_ms**: indica il tempo impiegato dall'algoritmo per calcolare il cammino minimo, espresso in millisecondi. Questo valore è utile per confrontare le prestazioni su griglie di diversa dimensione o complessità. L'analisi dei tempi permette di valutare l'efficacia delle ottimizzazioni implementate nel motore di calcolo del percorso.

10. **istr16**: contatore interno relativo a un particolare tipo di istruzione o operazione effettuata dall'agente durante il calcolo del cammino. Per il significato specifico, si rimanda alla documentazione delle ottimizzazioni descritte nella sezione 5.1.

11. **cont_frontiera**: indica il numero di nodi aggiunti alla frontiera di ricerca durante l'esplorazione della griglia. La frontiera rappresenta l'insieme delle celle candidate per il passo successivo del cammino. Questo valore consente di analizzare l'efficienza della gestione della frontiera e l'impatto delle eventuali ottimizzazioni sulla riduzione dei nodi visitati.

12. **nodi_salvati**: rappresenta il numero di nodi che sono stati memorizzati durante l'esecuzione dell'algoritmo per eventuali controlli o ricerche successive. Questo campo è correlato all'ottimizzazione della memoria e alla gestione delle sequenze di cammino, rimandando anche qui alla sezione 5.1.

13. **memo_usata**: indica la quantità di memoria heap utilizzata durante l'esecuzione del calcolo del percorso. Il valore è espresso in kilobyte (KB) e consente di valutare

l'impatto delle ottimizzazioni legate alla gestione della memoria, come ad esempio l'uso della memoria per sequenze parziali o versioni ottimizzate degli agenti.

14. **landmarks**: contiene la sequenza completa delle celle percorse dal cammino minimo trovato, rappresentata come stringhe separate dal carattere `|`. Questo campo è principalmente di supporto e permette di farsi un'idea sul percorso seguito dal Cammino Minimo o per eseguire ulteriori analisi. Poiché la sequenza può essere molto lunga, viene generalmente utilizzata solo per individuare ed eliminare errori o per generare mappe grafiche della soluzione.

Nota importante: alcuni campi, come `istr16`, `cont_frontiera` e `nodi_salvati`, sono direttamente influenzati dalle scelte di ottimizzazione del calcolo del cammino minimo, come il riordino della frontiera, l'uso della memoria per sequenze parziali, la versione ottimizzata dell'algoritmo, e altre strategie di miglioramento delle prestazioni. Per una spiegazione dettagliata delle implicazioni di ciascun parametro e del loro effetto sui valori riportati nel CSV, consultare la sezione 5.1 del manuale.
In sintesi, il file `report.csv` fornisce una panoramica completa delle caratteristiche delle griglie, delle prestazioni dell'agente e delle metriche di memoria e calcolo, consentendo all'utente di effettuare confronti sistematici tra test differenti e valutare l'impatto delle ottimizzazioni implementate.

8.6 Modalità Grafica (GUI)

La modalità grafica offre un ambiente interattivo per:

- creare griglie;
- modificarle manualmente;
- selezionare origine e destinazione;
- visualizzare il cammino;
- salvare e caricare file.

Creazione della Griglia

La finestra **Creatore Griglia** consente di:

- scegliere righe e colonne (max 100x100);
- selezionare il pattern di ostacoli;
- impostare un seme casuale;

- creare griglie personalizzate manualmente;
- caricare griglie da JSON.

Pattern Disponibili

- Densità bassa
- Densità media
- Densità alta
- Spirale
- Scacchiera
- Righe
- Unico ostacolo
- Personalizzata

Per ulteriori informazioni relative alle tipologie di griglia, consultare la sezione 1 "2"
Una volta che si ottiene una combinazione di pattern, dimensioni e seed desiderata, o se si vuole solamente visualizzare la configurazione attuale, selezionare "Aggiorna anteprima".
Quando si desidera, selezionare "Conferma", per poter interagire con la griglia.

Interazione con la Griglia

Nella finestra principale:

- clic sinistro per selezionare origine (O);
- secondo clic per destinazione (D);
- clic successivo cancella O e D selezionate e imposta una nuova origine.

Avvio del Calcolo

Alla pressione di **Avvia valutazione**:

- si apre una finestra di configurazione;
- è possibile impostare le stesse opzioni della CLI;
- l'esecuzione avviene in background;
- il pulsante **Termina valutazione** consente l'interruzione.

Riepilogo Finale

Al termine:

- viene mostrato un riepilogo testuale;
- è possibile salvarlo in formato `.txt`.

8.7 Salvataggio e Caricamento

Salvataggio JSON

Le griglie possono essere salvate in formato JSON, includendo:

- ostacoli;
- origine e destinazione;
- dimensioni.

Caricamento JSON

Il sistema:

- valida le dimensioni;
- carica automaticamente la griglia;
- aggiorna l'anteprima.

8.8 Ottimizzazioni dell'Algoritmo

Questa sezione descrive in modo teorico e concettuale le **ottimizzazioni disponibili** durante il calcolo del cammino minimo.
Tutte le opzioni selezionabili in CLI e GUI fanno riferimento a questa sezione.

Riordino della Frontiera

Ottimizza l'esplorazione dei nodi migliorando la priorità di espansione.

Condizione Forte

Introduce criteri più restrittivi per la convergenza del cammino.

Memoria delle Sequenze

Consente il riuso di sequenze parziali già esplorate.

Versione Ottimizzata

Attiva implementazioni alternative più efficienti in termini di costo computazionale.

Nota: Per evitare ridondanze, il comportamento dettagliato di ciascuna ottimizzazione non viene ripetuto nelle sezioni operative, ma va sempre riferito a questa sezione.

Elenco delle figure

1 Esempio di griglia con pattern casuale a densità bassa. 5
2 Esempio di griglia personalizzata definita dall'utente. 5
3 Esempi di pattern automatici. 6
4 Esempi di generazione di griglia con pattern deterministico e casuale. . . 10
5 Esempio di evidenziazione del contesto (verde) e del complemento (arancione) di una cella O. 13
6 Esempio di evidenziazione delle celle di frontiera tramite un cerchio contenuto in alcune celle facenti parti del contesto o del complemento. 15
7 Esempio di visualizzazione grafica del percorso minimo generato da `CamminoMin` e a lato il riassunto con i dati relativi all'esecuzione. 23
8 Tempi di esecuzione per griglie 25×25 29
9 Memoria utilizzata per analizzare griglie 25x25 32
10 Celle di frontiera visitate per griglie 25x25 35
11 Rapppresentazione grafica della tabella 4 37

Elenco delle tabelle

1 Tempi di esecuzioni medi misurati in ms per griglie 25×25 per diverse modalità di esecuzione di `CamminoMin()` 28
2 Utilizzo della memoria medio (Heap) misurato in KB per griglie 25X25 per diverse modalità di esecuzione di `CamminoMIn` 31
3 Celle di frontiera analizzate (media) per griglie 25X25 per diverse modalità di esecuzione di `CamminoMIn` 34
4 Tempi di esecuizione in ms per la modalità di ottimizzazione migliore R-MV 37

Elenco degli algoritmi

1 generaOstacoli(Griglia, p, s) 9
2 camminoLibero(griglia, origine, destinazione, tipo) 12
3 percorri(griglia, origine, passoX, passoY, passi) 13
4 controllaVicini(cella, griglia) 14
5 camminoMin(griglia, O, D) 17
6 SalvataggioRisultatoIntermedio(camminoCorrente) 20
7 RiordinaFrontiera() 21
8 Funzioni per la gestione della memorizzazione dei Cammini già Visitati . . 22